

Osnabruck University

Institute for Cognitive Science

Bachelor's Thesis in Cognitive Science

Dynamic Adjustment of Monte Carlo Tree Search Simulations for A 9×9 -Go-AI

Author:	Moritz Richter
1st Examiner:	Tobias Thelen
2nd Examiner:	Ulf Krumnack
Start Date:	10.03.2026
Submission Date:	29.06.2026

Contents

1	Introduction	4
2	The Rules of Go	5
3	Historical Evolution	6
3.1	Before Deep Learning	6
3.1.1	Before MCTS	6
3.1.2	After MCTS	7
3.2	Deep Learning	7
3.2.1	The Alpha-Models	7
3.2.2	MuZero	8
3.2.3	Gumbel AlphaZero	8
3.2.4	Further research	8
3.2.4.1	KataGo	8
3.2.4.2	Dynamic MCTS	9
4	Technical Background	9
4.1	Supervised Learning	9
4.2	Reinforcement Learning	9
4.3	Self-Play	10
4.4	Neural Networks	10
4.4.1	Artificial Neural Networks	10
4.4.2	Convolutional Neural Networks	11
4.4.3	Residual Neural Networks	11
4.5	Monte Carlo Tree Search	12
4.6	Selection Methods	13
4.7	AlphaGo	14
4.8	AlphaGo Zero	15
4.8.1	Structure of AlphaGo Zero	15

4.8.2	Training of AlphaGo Zero	16
4.9	Kendall Rank Correlation Coefficient	16
4.10	Elo	17
5	Methodology	17
5.1	Reasoning 9×9 vs. 19×19	18
5.2	Choosing AlphaZero Over Alternatives	18
5.3	Environment	18
5.3.1	Base Implementation	18
5.3.2	Computational Resources: University HPC Cluster	18
5.4	Heuristics	19
5.4.1	Entropy of Likeliest k Actions	19
5.4.2	Comparison of High And Low Ranking Moves	20
5.4.3	Auxiliary Head Estimating Uncertainty	21
5.4.4	Random	22
5.5	Evaluation	23
6	Results	23
7	Discussion	25
8	Conclusion	27
	Bibliography	28

1 Introduction

Creating a machine that is capable of playing games has fascinated humanity [1] for a long time and been an integral part of computer science [2] since its inception. It is a benchmark for human intelligence and explores the boundaries of what is possible using automatic decision making. A major milestone was reached with the defeat of reigning world-chess champion, Garry Kasparov, at the computations of the IBM chess computer DeepBlue in 1997 [3]. This event spurred on further improvement in chess machines that is still ongoing [4] and research on a multitude of other games. One such game was Go, which remained an unbeaten challenge to scientists for 19 more years [5]. While many approaches were tested, it was not until 2016 that, on equal footing, a high-ranking professional player was beaten by Google-DeepMind's artificial intelligence (AI), AlphaGo. AlphaGo used a vastly different approach from earlier game computers and programs [6]. It combined deep neural network (DNN) training with Monte Carlo Tree Search (MCTS), that is, state exploration guided by learned policy networks. This work builds upon AlphaZero, a generalisation of AlphaGo that achieves high performance without human knowledge.

MCTS based models like AlphaZero make use of a high number of simulations during their search. However as such, MCTS requires large amounts of computational power. This follows a trend in modern AI research and systems of growing requirements on resources that are necessary for training as well as for their subsequent deployment [7]. In the case of MCTS, simulations are executed even though the best action might already have the highest number of visits.

This motivates the central claim of this thesis that the efficiency of MCTS augmented with predictors can be increased by leveraging the predictor's values to dynamically adjust the number of simulations without degrading performance in playing strength. The development and implementation of such predictive heuristics are the goal of this thesis. In the end, these heuristics will be evaluated by playing against each other.

During the course of this thesis, first, the theoretical background for this thesis is presented.

Then the evolution of Go-AIs is described. Afterwards, the methods of this project are explained. Next, I show the results of the application of the developed heuristics. The heuristics show improved capability on the base model.[CHECK](#) Lastly, the thesis is summarised.

2 The Rules of Go

Go is a board game that originated in China with a history spanning more than 4,000 years. It is played by two-players using black and white stones. While usually played on a 19×19 -grid, other board sizes like 9×9 or 13×13 are also common. The goal of the game is to enclose more territory, meaning, grid intersections, than the opponent.

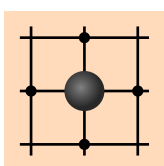


Figure 1: Liberties of stones

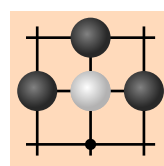


Figure 2: Capturing a Stone with
the last liberty

The players place stones in turn; the player with the black stones begins. The group of free places around a single stone are called liberties, see Figure 1. Stones of the same colour that are placed on neighbouring fields are considered a group and share the same liberties. A group may be arbitrarily large. A group of the opponent can be captured by filling in all liberties of the group, see Figure 2, which removes them from the board. While generally any stone may be placed anywhere on the board, it may not be placed in places where the stone or its group is captured because of the stone having been placed. The exception is if the stone is the cause for a successful capture of the opponent's stones. Importantly, a board position may not be immediately repeated. This means, after a stone has been captured, the capturing stone may not be recaptured on the next move. This is called Ko. The game ends when both players pass consecutively, typically when all meaningful moves have been played and the territories are settled. Once the play is concluded, the territory is counted. Unoccupied grid intersections within their territory each correspond to one point for the respective player. Captured stones each represent additional points for the player who captured them. Captured

stones that remain on the board while counting are worth two points for the occupied field and the stone itself.

To compensate black's advantage in going first and avoid draw, additional 5.5-7.5 points are awarded to white. This is called komi. Komi can be adjusted to even out the differences in capability between inequally strong players. Lastly, komi can be combined with handicap stones with which black stones are placed on predetermined spots on the board before the start of the game. In this case, the white players places the first stone.

Traditional ranks in Go are called kyu and dan, with an additional consideration for professional players. Beginners' ranks start at 30 kyu reaching until 1 kyu. After reaching 1 kyu, players can advance from 1 dan to 9 dan. Professional players are ranked separately with their own dan-scale. Nowadays, players are also evaluated using the Elo-system [8].

3 Historical Evolution

3.1 Before Deep Learning

3.1.1 Before MCTS

Early Go programs were usually rule-based and used pattern recognition combined with heuristic evaluation functions and search to find hard-coded patterns and choose actions based on the results.

One such program is "The Many Faces of Go" that was using a large compilation of rules and an influence function [9]. It was competing successfully in Computer Go championships in 1998-2002. It was widely considered one of the best Go programs of the time [10]. At the time, it reported online strength estimates at 8 kyu, corresponding to an Elo value of circa 1200. It has since been continuously improved and made to embrace new technologies until the introduction of deep learning [11].

A freely consultable open-source program of the time that also works based on rules and pattern-recognition is GnuGo [12]. Shortly, within the pipeline, GnuGo first analyses the board for its state, examines groups, their life-and-death, and their relation to one another. Next, it

generates the moves, and lastly evaluates the moves for its effect on influence and territory. The move with the highest score is played.

3.1.2 After MCTS

Earlier Go programs were found to perform well in specific parts of the game like life-and-death problems or local tactical analysis [10]. However they were unable to play a complete game well or even better than amateur players. Programs of this level of sophistication could not achieve the same level of success for Go as for chess, because the computational search for Go is drastically different. While it is not the only factor, the state-space is much larger on a large board, $3^{19 \times 19} \approx 10^{170}$ of which 1.2% are legal. However, Go on a 9×9 -board has a similar branching factor to chess and has proved equally as difficult as Go on a large board. This is due to static position evaluation requiring a large amount of local auxiliary computations. Additionally, in order to find legal moves, the recent history of the game has to be contained within the state information to account for special rules such as Ko that forbid repetition.

As the typical game is also much longer than in chess, MCTS is a tool better suited to explorations because it is capable of a more dynamical approach [13]. Furthermore, short-term profit may often be outpaced by an opponent's stronger tactical play. With the introduction of MCTS into the field of Computer Go, programs became improved significantly.

In Go on a 9×9 -board, a 5 dan professional player was beaten by MoGo in 2007; in 2009, a 9 dan professional by Fuego [14]. In the course of pure MCTS applied to Go on a 19×19 -board, various models managed to defeat high-ranking professional players, albeit with up to 9 handicap stones, and gained a ranking of 4 dan on online play.

3.2 Deep Learning

3.2.1 The Alpha-Models

AlphaGo revolutionised the field by combining DL with MCTS. The algorithm is explained in a later section. There are multiple named training runs of AlphaGo, each improving on the previous. "AlphaGo Fan", which was the program to win against a human professional

on a 9×9 board, won 5-nil [6]. “AlphaGo Lee Sedol” won against Lee Sedol with a record of 4 – 1. The last iteration, “AlphaGo Master”, won against top human professionals in online games with a record of 60 – 0 [15]. When these models were compared to AlphaGo Zero, their Elo ratings were 3, 144, 3, 739, and 4, 858, respectively. AlphaGo Zero received a rating of 5, 158 [15]. AlphaZero is another iteration on this self-play algorithm [16]. While ultimately the same structure as AlphaGo Zero, the domain-specific optimisation that Go is rotationally invariable is removed which reduces the amount of data of data available for this game. On the other side, this allows AlphaZero to be used for more games than just Go. The algorithm is explained in a later section.

AlphaZero managed to beat state-of-the-art models in chess (Stockfish), shogi (Elmo), and Go (AlphaGo Zero).

3.2.2 MuZero

MuZero is a generalisation of the Alpha-family [17]. While previous models required a working model of the environment, MuZero is capable of learning about the environment through interaction with the environment. While also achieving similar results on traditional games such as Go and chess, it is also capable of learning to play real-time games like Atari. In all of these domains, it was capable of surpassing state-of-the-art models and therefore top human professionals.

3.2.3 Gumbel AlphaZero

Google DeepMind’s successor model, Gumbel AlphaZero, breaks from traditional MCTS using UCB, the basis for UCT [18], or variants, and instead makes use of other policy improvement strategies [19]. One such strategy is sequential halving, in which the window of considered moves is halved considering from the highest-rated moves.

3.2.4 Further research

3.2.4.1 KataGo

KataGo represents a significant step to the wider Computer Go community [20]. While similarly strong models such as FineArt, a proprietary Chinese model, or LeelaZero, a “fairly faithful reimplementa-tion” of the original AlphaGo Zero model [21], exist, KataGo is an open-

source model employing additional strategies. Improving on the architecture and process of AlphaZero, KataGo introduces and publicly explains several domain-dependent and -independent additions to improve efficiency, flexibility, and utility for human analysis.

Some of these features include:

- Auxiliary policy targets: In addition to simply predicting the best move for the current board position, KataGo also predicts the opponent's move.
- Playout cap randomisation: Most board positions are searched with a low number of simulations and only used to train the value head, the head that evaluates the likely winner of the position. Long playouts are then used for policy training.
- Blocking overplay: Forbidding playing in a certain region after a number of moves contributes in efficiency, albeit little in regard to performance.
- Auxiliary ownership and score targets: The network also predicts to whom an intersection belongs as well as a estimations of the point difference at the end of the game.

3.2.4.2 Dynamic MCTS

Forming a basis for this thesis, Lan et al. introduced estimations of uncertainty at intervals during training, to determine whether continued search is necessary [22]. As MCTS finds the optimal move after 1 simulation 62% of the time, and most positions require far less than the maximal number to find the optimal move, it is permissible to explore fewer situations. Thus, if there is founded certainty that the best move has been found, search should terminate.

4 Technical Background

4.1 Supervised Learning

Supervised learning (SL) describes a machine learning (ML) paradigm in which an agent is taught to recognise patterns from labeled data examples.

4.2 Reinforcement Learning

In contrast to SL, Reinforcement Learning (RL) is a collection of methods with which an agent is trained to maximize a reward signal in interaction with its environment [23]. Generally during training, an agent makes use of two concepts called exploitation and exploration. On

one side, exploitation is choosing a path of action that is known to produce a good result. On the other side, exploration is choosing a path of action that is not well-known and thus, upon further examination, may produce a better result than already known paths of action.

4.3 Self-Play

Self-play is a training scheme within the field of RL [15]. In self-play, an algorithm competes against an earlier iteration of itself. In the scenario of this paper, a training version plays a match of Go against a previous iteration of itself. In this sense, the previous iteration is part of the environment and the reward signal is the outcome of the match. Various selection strategies exist for the iteration against which is played, for example, a random iteration is chosen to play, or the selection follows a “King-of-the-Hill”-approach, where the strongest iteration remains as the benchmark and is to be beaten. This, more than the randomised selection shows growth in performance of the agent.

4.4 Neural Networks

4.4.1 Artificial Neural Networks

Currently, artificial neural networks (ANN) are one of the most used and researched technologies. After their groundwork was laid in 1958 by Rosenblatt [24] with the invention of the perceptron, new methods have been since been developed for a large number of purposes, not the least of which is playing games and improving strategies to superhuman levels.

One of the simplest structures of ANNs is the feedforward neural network consisting of multiple layers of many stacked neurons. In this network, every neuron receives an input from all neurons in the preceding layer plus a bias individual per neuron. This is also known as dense layers. ANNs receive an input of arbitrary format, defined per usecase. During the forward pass, the signal of this input is propagated through the network to the output layer [25]. One layer is made up of multiple neurons, each with its own weights per input, plus a bias individual to each neuron. During RL, the ANN’s output is evaluated for its reward signal in response to the environment. From this, an error signal is generated and passed backwards through the network. Each layer is updated dependent on the succeeding layer’s, error signal.

4.4.2 Convolutional Neural Networks

Most objects of the physical world depend on their multi-dimensional structure to be considered when analysing them. Images, for example, rely on the spatial position of pixels in relation to one another in order to correctly depict objects. Humans make use of layered groups of neurons with distinct tasks to recognise groups of features and form an understanding of objects from them.

Computationally, this structure is approximated using Convolutional Neural Networks (CNN) that uses convolutional layers alongside or instead of dense layers. Similarly to their biological counterparts, CNNs find substructures that are common for multiple objects in early layers [26]. Combining these features in later layers enables recognition of more specific types of objects. In a convolutional layer, kernels, that is, tensors, operate on a receptive field of inputs to generate outputs by sliding over the input making the operation translationally invariant instead of operating on the whole input at the same time. The weights in these layers can be learned through training.

4.4.3 Residual Neural Networks

In the course of research and development of ANNs, they have grown exponentially deeper. Beginning with single perceptrons, which were used in a single layer, modern networks use neurons with more than 100 layers and a large number of perceptrons per layer. Users of ANNs of this size noticed the problem of the vanishing gradient [27].

To solve this problem, residual neural networks (ResNet) were introduced in Deep Learning (DL) problems. ResNet's make use of "skipping connections" [27]. In one of connection, a residual layer projects its output not only to the succeeding layer but also to another layer further down the stack of layers, skipping the layers inbetween. The residual output is added to the output of the layer, to which it is projected. This largely fixes the vanishing gradient since the output of the layer is passed further into the stack of layers.

A negative point in this setup is that the intermediate layers may be found to be superfluous and converge to 0 during training but still demand the necessary effort for their computation.

Go, being a board game that is played in two-dimensions, is especially well-suited to being analysed by CNNs. Much like recognising an object, situations such as fights or life-and-death, in which, for example, two or more often neighbouring groups compete for life, that cause a local effect can be analysed in early layers and evaluated within the global context of the match in later layers. For proper recognition and evaluation of local and global situations, Go requires deep insight of the board. For more accurate representations of the global position, the combination with ResNets therefore becomes vital to sustain trainability in growing networks.

4.5 Monte Carlo Tree Search

MCTS is a versatile heuristic decision making algorithm. During the search, a tree is built based on the original state represented by root node R . Every child node i of R is a possible subsequent state from the state of R , in which an action a_i was taken. Through exploration of subsequent state from R , MCTS finds a state that is most likely to maximise a metric that can be changed per use-case. Every node i of the tree keeps track of how many simulations made use of i , n_i , and how many simulations are considered wins, w_i . There are four stages to MCTS, the process is graphically presented in Figure 3:

1. Selection: From R , choose a child node according to a selection method, often a variant of UCT [18]. Repeat until choosing a leaf node C .
2. Expansion: If C is not a terminal state, which in Go is a state in which both players have passed and playing does not continue, create new child nodes. Choose a new child state S according to the selection method.
3. Evaluation¹: Evaluate S . S may be evaluated by choosing a child state using an rollout policy, e. g., randomly or a quick decision making unit [6], until a terminal state which has a definitive value. Another method is an evaluation function applied to S [15].
4. Backpropagation: Update the visit and win counts in the path from S to R under consideration of the result from the evaluation.

¹This step is also called simulation. For the sake of clarity within the scope of this thesis, “evaluation” will be used for this step of the process.

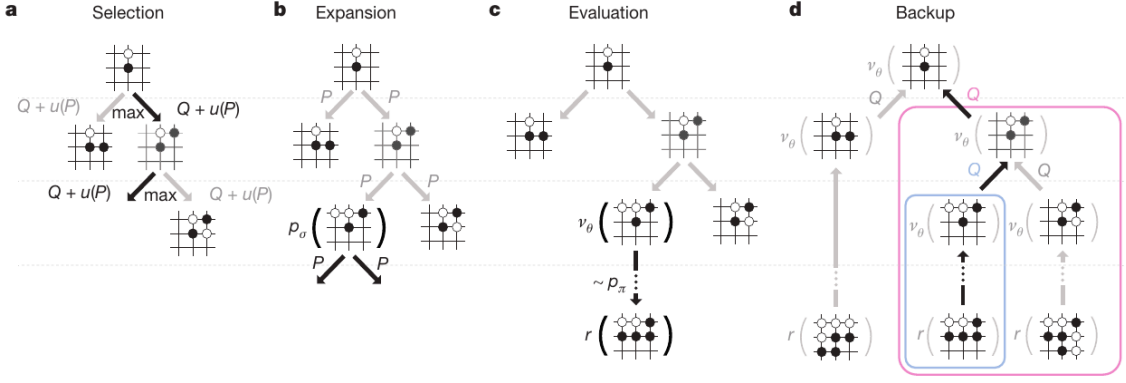


Figure 3: Illustration of MCTS from Silver et al. [6]

Through rigorous exploration of the tree by simulations, meaning, paths from root to terminal nodes, the child states that most likely maximise the evaluation method are expected to converge to higher values in the selection method.

In the context of AlphaZero, the states being represented by the nodes are board positions in which moves are played as the selected actions that lead to child nodes or subsequent board positions.

MCTS returns a probability distribution in which better moves according to the evaluation method have a higher evaluation in terms of the selection. This probability distribution can be used as policy to choose moves in a play scenario.

4.6 Selection Methods

One selection method that is commonly used in MCTS's selection phase is the "Upper Confidence Bound 1 applied to Trees" (UCT) [6], [18]. It is used to find a balance between exploitation of well-simulated moves and exploration of under-explored moves in MCTS. In the selection stage, node i with the highest value according to the expression

$$Q(s, a_i) + U(s, a_i) = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}} \quad (1)$$

is selected, where

- $Q(s, a) = \frac{w_i}{n_i}$ is a measure of how good a_i is in state s ,
- $U(s, a) = \sqrt{\frac{\ln N_i}{n_i}}$ is a measure of how well explored move a_i is,
- w_i is the number of wins that the successor states of s accumulate if a_i is played,
- n_i is the number of times that a_i was chosen during the simulations,

- N_i is the number of times that s was part of the simulations,
 - c is an exploration parameter. The higher c , the more explorative the move selection is.
- Since $N_i \geq 0$, $\sqrt{\frac{\ln N_i}{n_i}} \approx \infty$ if $n_i = 0$. Consequently, every move is tried out at least once.

The Probabilistic Upper Confidence Bound applied to Trees (PUCT) is a variant of UCT that incorporates prior estimations on the goodness of the actions of a state [15]. These evaluations are typically provided by predictors like a neural network or domain-specific heuristics.

It can use different uncertainty formulas, for example:

$$U(s, a) = c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}, \quad (2)$$

where

- c_{puct} is a constant that guides the level of exploration and
- $P(s, a_i)$ is the evaluation of move a_i in state s according to the predictor.

Since $1 + N(s, a) \gg \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \ll \infty$. In effect, unlike in UCT, not every move is forced to be simulated at least once, even though unlikely in practice.

4.7 AlphaGo

The AlphaGo-algorithm is a combined system of deep CNNs, MCTS, SL, and RL [6]. SL policy network p_σ was trained from database of expert-level moves. The fast rollout policy p_π was designed to quickly execute reasonable moves until a final state in the evaluation step of MCTS. RL policy network p_ρ on the other hand is trained to optimize the outcome of self-play.

The RL value network $v^{p(s)}$ predicts the outcome based on the dataset created by p_ρ 's selfplay.

Supervised learning. The first policy network $p_\sigma(a|s)$ receives as input a board representation s and returns a move probability distribution over all legal moves a . It is optimised to correctly predict the selection of move a , given s , using stochastic gradient descent according to $\Delta\sigma \propto \frac{\partial \log p_\sigma(a|s)}{\partial \sigma}$.

Reinforcement learning. The second policy network $p_\rho(a|s)$ is architecturally the same. Its weights are initialised to the weights of the p_σ and further trained using policy gradient reinforcement learning. Self-play occurs between the current and a random previous iteration

of the network. The reward function is zero for all non-terminal time steps and either $+1$ or -1 for winning and losing, respectively. The gradient is calculated to $\Delta\rho \propto \frac{\partial \log p_\rho(a_t|s_t)}{\partial \rho} z_t$.

The value network $v_\theta(s)$ approaches the result from perfect play $v^*(s)$ by approximating the result found through using the best policy of p_ρ , $v^{p(s)} = \mathbb{E}[z_t | s_t = s, a_{t...T} \sim p]$.

Architecturally, v_θ is similar to the policy networks but is trained on state-outcome pairs (s, z) that are gathered during p_ρ 's selfplay under stochastic gradient descent to minimize the mean-squared error (MSE) between predicted value $v_\theta(s)$ and the actual outcome z $\Delta\theta \propto \frac{\partial v_\theta(s)}{\partial \theta} (z - v_\theta(s))$.

Search In MCTS, the selection follows PUCT. In the expansion, the prior probabilities $P(s, a)$ for leaf node s_L are provided by p_ρ ; the evaluation $V(s_L)$ is generated by combining the value network's output $v_\theta(s_L)$ and the fast rollout's output z_L through a weighting constant λ :

$$V(s_L) = (1 - \lambda)v_\theta(s_L) + \lambda z_L. \quad (3)$$

From the visit distribution after MCTS, the action with the most visits is chosen.

4.8 AlphaGo Zero

AlphaGo Zero represents a significant improvement over its predecessor, achieving better performance through a more efficient architecture [15]. By eliminating the need for human expert data, it masters the game entirely through self-play.

4.8.1 Structure of AlphaGo Zero

Unlike AlphaGo that uses separate policy and value networks, AlphaGo Zero makes use of a single unified network, that consists of residual convolutional layers as well as batch normalisation and rectifier nonlinearities. This network, $f_\theta(s)$, takes as input a board state s and outputs both move probabilities and state evaluation:

$$(\mathbf{p}, v) = f_\theta(s). \quad (4)$$

The probability distribution $\mathbf{p}$ is a vector over all allowed moves in s , where $p_a = P(a | s)$.

The scalar value $v \in [-1, 1] \approx \mathbb{E}[z, s]$ is the expected outcome of the game, where z is the outcome of the game. If z is equal to 1 , the current player wins. If z is equal to -1 , the opponent wins.

The distribution $\mathbf{p}$ and scalar v guide the following MCTS in the simulations. To avoid the agent narrowing in on suboptimal solutions, Dirichlet-noise is added. A Dirichlet distribution $\eta, \eta \sim \text{Dir}(0.03)$, is added to $\mathbf{p}$: $P(s, a) = \varepsilon \eta_a + (1 - \varepsilon) p_a$, weighted by ε .

At each step during the simulation, the algorithm selects the action a_t that maximises PUCT using the predicted probability and value from the network. After the allotted simulations, the MCTS-returned visit distribution is used to create a new distribution

$$\pi(a|s_0) = \frac{N(s, a)^{\frac{1}{\tau}}}{\sum_b N(s_0, b)^{\frac{1}{\tau}}}, \quad (5)$$

where τ is a temperature variable that controls exploration. Towards the end of training τ is made to approach 0. This changes the resulting distribution from a being flat to spiking. From $\pi(a|s_0)$, the action is sampled.

4.8.2 Training of AlphaGo Zero

AlphaGo Zero learns through an iterative self-play loop. The current best version, whose weights are denoted by θ , plays against itself to generate a dataset of (s_t, π_t, z_t) , that is, the state, the MCTS distribution, and the final value of the game that the state comes from. The network is updated to minimise the composite loss function

$$l = (z - v)^2 - \pi \log \mathbf{p} + c \|\theta\|^2. \quad (6)$$

Furthermore, the fact that Go is rotationally invariable is exploited by rotating the board and thus generating more valid game positions.

4.9 Kendall Rank Correlation Coefficient

The Kendall Rank Correlation Coefficient (KRCC) is a tool to calculate the similarity of the same type of objects [28]. In the context of this thesis, objects are AlphaGo Zero’s predictor network’s move probability distribution $\mathbf{p}$ and MCTS’ node frequency distribution π . Positions within these distributions are always examined together. In the implementation used in this thesis, all positions are iterated and compared to all following positions. A pair of positions is concordant if both distributions agree either that $\mathbf{p}_i < \mathbf{p}_j$ and $\pi_i < \pi_j$ or that $\mathbf{p}_i > \mathbf{p}_j$ and $\pi_i > \pi_j$. This thesis uses specifically KRCC’s variant τ_b because equal values are possible in either move probability distribution. In that case, a pair is tied if $\mathbf{p}_i = \mathbf{p}_j$ or

$\pi_i = \pi_j$, both may occur at the same time. Otherwise, the pair is said to be discordant. The formula is

$$\tau_B = \frac{n_c - n_d}{\sqrt{(n_0 - n_1)(n_0 - n_2)}}, \quad (7)$$

where

- n_0 is the sum of all pairs, $n_0 = n(n - 1)/2$,
- n_1 is the sum of tied values in $\mathbf{p}$, $n_1 = \sum_i t_i \frac{t_i - 1}{2}$,
- t_i is the number of tied values containing the i -th unique value,
- n_2 is the sum of tied values in π , $n_2 = \sum_j u_j \frac{u_j - 1}{2}$, and
- u_j is the number of tied values containing the j -th unique value.

4.10 Elo

The Elo rating system is used to rank players of games such as Go and Chess. It considers the likelihood of one player to win over their opponent [8]. The central formulas are

$$E_A = \frac{1}{1 + 10^{\frac{R_B - R_A}{400}}}, \quad (8)$$

which calculates the expected result of player A with rating R_A against player B with rating R_B , and

$$R'_A = R_A + K \cdot (S_A - E_A), \quad (9)$$

which updates the ranking of player A . S_A is the actual result and can be 1 for a win, 0.5 for a draw, and 0 for a loss. Established players are attributed a lower factor K to stabilize rankings, whereas newer players have a higher factor K to allow rankings to converge more quickly to their real ranking by increasing the gained points' coefficient.

Elo points can give an intuition on how likely one player is to win over the other. For example, player A with 400 points more than his opponent B in the Elo system should win 10 games out of 11 because from $R_B - R_A = -400$ follows $\frac{1}{1 + 10^{-1}} = 91\%$.

5 Methodology

In the following, possible heuristic approaches to dynamically lower the number of MCTS simulations will be presented. These methods will be trained separately for individual

networks. The resulting models are evaluated by matching them against each other to study playing strength and the actually used number of MCTS-simulations.

5.1 Reasoning 9×9 vs. 19×19

A 9×9 board has a state-space complexity of $3^{81} \approx 4 \times 10^{38}$, of which 23.43% $\approx 1.04 \times 10^{38}$ [29] are legal. In contrast, a 19×19 board has $3^{19 \times 19} \approx 1.74 \times 10^{172}$ possible positions and 2.082×10^{170} legal positions. Thus, in an effort to gain comprehensive results within the hardware and temporal limits of this thesis, the board size is restricted to 9×9 .

5.2 Choosing AlphaZero Over Alternatives

The basic AlphaZero structure is used as the baseline in this work for reasons of clarity and experimental control. Although models such as KataGo achieve state-of-the-art performance [20], it does so by incorporating a wide range of engineering optimisations, training heuristics, and system-level improvements. As a result, isolating the effect of modifications within such a system can be difficult, since performance gains may arise from interactions with existing components rather than the modification itself.

Despite the relative successes of the Gumbel models compared to their PUCT-based predecessors [19], the PUCT-based model still remains a widely employed approach in popular Go models [20].

5.3 Environment

5.3.1 Base Implementation

The primary research tool for this thesis is Michael Hu’s implementation [30]. It provides a robust implementation of the AlphaZero algorithm with native support for Go using the Chinese ruleset and tools for model analysis. It is fully written in Python and relies on PyTorch.

5.3.2 Computational Resources: University HPC Cluster

Due to the high computational cost of self-play reinforcement learning, all training and evaluation experiments are conducted on Osnabrück University’s High-Performance Computing (HPC) Cluster².

The cluster provides multiple GPU-enhanced nodes. Each relevant node includes

²Project number of the HPC: 456666331

- 4 NVIDIA A100 GPUs,
- 128 AMD EPYC CPUs, and
- 927.7 GB of memory.

Resources are managed by the SLURM workload manager operating on Rocky Linux 8.5.

5.4 Heuristics

The heuristics suggested here are based on the idea, that the network’s predictions is able to express an inherent certainty that can be leveraged to adjust the amount of simulation needed to generate a successful training sequence while lowering the computational efforts.

Here, MCTS augmented with predictors and heuristics uses the reference number of simulations to recalculate a new number of simulations. These calculations and their rationale are explained in the following.

5.4.1 Entropy of Likeliest k Actions

The goal for this heuristic, top_k_az , is to calculate the entropy of the k highest-rated moves of the predictor’s move probability distribution and adjust the amount of simulations accordingly.

Entropy is by itself a measure of uncertainty for a distribution of probabilities. The use of the entropy is therefore an apparent possible solution to the goal of this thesis. Furthermore, using the 2 likeliest options of a probability distribution is a common approach to gather a measure of certainty of that distribution. Increasing this window of consideration should give a more balanced view.

The important formulas are

$$H_{\text{norm}} = \frac{H(\text{Top}k)}{\ln(k)}, \quad (10)$$

$$n_{\text{new}} = n_{\text{base}} \cdot \frac{2}{1 + e^{-s(H_{\text{norm}} - sp)}} \quad (11)$$

where

- $H(\text{Top}k)$ is the entropy of the k highest ranked moves of π that are sorted descendingly in $\text{Top}k$.
- H_{norm} is the result of re-normalising the entropy of these moves to within the range $[0, 1]$,

- n_{new} is the applied number of simulations,
- n_{base} is the baseline number of simulations,
- s is the steepness of the graph, and
- sp is the position of the inflection point on the x -axis.

Equation 11, represented in Figure 4, is a logistic function. The formula here will continue to be used with $s = 8$ and $sp = 0.5$. The graph is maximised at $\frac{2}{1+e^{-8(1-0.5)}} \approx 1.964$ and, because H is normalised, it is minimised at $\frac{2}{1+e^{-8(0-0.5)}} \approx 0.036$. These values should allow the entropy of the k highest-rated moves to transition fluidly around the inflection point at 0.5.

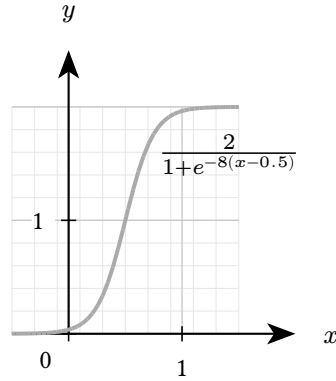


Figure 4: Logistical uncertainty function for high-rated moves' entropy.

Effort for this heuristic should be negligible. The algorithmic complexity of finding the highest-sorted is $O(|A| \cdot \log k)$, with $|A|$ as the space of all actions. There also is the additional cost of calculating the entropy of $\text{Top}k$.

5.4.2 Comparison of High And Low Ranking Moves

This heuristic, `top_bot_k_az`, measures the disparity between the probability densities of the k highest-rated moves and the $A - k$ lowest-rated moves. Using this distance, the number of simulations is adjusted.

The above heuristic has a strong focus on relatively few high-rated moves compared to the overall space of possible moves. This increases vulnerability to overconfidence where it is not warranted. That is especially the case when a single move is disproportionately well-rated. Therefore, one solution to this problem can be a more global consideration of the move probability space.

$$d = \sum_{a \in \text{Top}k} \pi(a) - \sum_{a \in A \setminus \text{Top}k} \pi(a), \quad (12)$$

$$n_{\text{new}} = n_{\text{base}} \cdot \frac{2}{1 + e^{s(d-sp)}}, \quad (13)$$

where

- $\pi(a)$ is the probability of a in π ,
- $\text{Top}k$ is the array of the highest-rated k actions.

Equation 13 is similar to Equation 11 with the entropy exchanged for the difference of the probability densities and the sign of the slope s flipped.

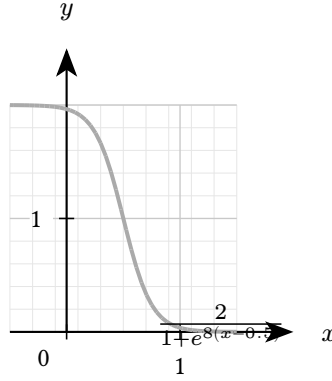


Figure 5: Logistical uncertainty function for High/Low Comparison.

This solution should be able to help mitigate the inequal consideration of moves from the first heuristic. Computational effort consists of the sorting in $O(n \log k)$ and the addition. Therefore the effort should be about as much as for the first heuristic.

5.4.3 Auxiliary Head Estimating Uncertainty

Lan et al. introduced supplementary networks to estimate certainty and stop searching when reaching a certain threshold. However, as merging networks for predicting value and policy has proved successful previously, enabling the network to predict its own prediction-search disagreement should also turn out beneficial. In this heuristic, `rl_az`, the predictor network f on board position s will return

$$f(s) = (\mathbf{p}, v, u), \quad (14)$$

where $\mathbf{p}$ is the move probability distribution, v the evaluation, and u the uncertainty estimation. The number of simulations is adjusted according to the formula

$$n_{\text{new}} = n_{\text{base}} \cdot \exp(u). \quad (15)$$

This means that higher uncertainty correlates with searching more thoroughly correlates while less uncertainty correlates with a lower number of simulations. The exponential function allows both to increase and decrease the number of simulations. n_{new} also is limited to a maximal number to stop the search from going unendingly long.

The loss will be calculated using the expression

$$l_{\text{all}} = l_{\text{az}} + c \cdot (u - \lambda \cdot \tau(\mathbf{p}, \pi))^2, \quad (16)$$

where

- l_{az} is the error signal used by AlphaZero considering $(\mathbf{p}, \sigma, v, \theta)$,
- c is a weighting constant for MSE,
- $\tau(\mathbf{p}, \pi)$ is the KRCC of board state s ,
- λ is a scaling constant for the KRCC, and
- π is the distribution found by MCTS.

In this context $\tau(\mathbf{p}, \pi)$ measures the similarity between the ranked move distribution estimated by the prior and the ranked move distribution determined by MCTS.

This heuristic should be capable of reducing the average simulation numbers by predicting how much move estimations change during MCTS in different positions. However, especially the process can be very time-intensive in the beginning of a single game when more moves are available and the calculation of KRCC requires more effort.

While the idea is promising, a glaring problem with this approach is the computational cost of calculating the KRCC for the two distributions. One solution is to similarly to the above heuristics consider only the highest k actions. This is also looked at in the following sections.

5.4.4 Random

Following the example of Wu et al. [20], a uniformly chosen number from $n \in [1, 200]$ will be used as the number of playouts for MCTS in order to investigate in comparison whether these strategies actually fulfill their purpose and have an active part in lowering computational effort in this heuristic, `rand_az`. If randomly adjusting the amount of simulations for MCTS performed equally well as the strategies suggested above and assuming that they all perform

similarly well to the base line model, this would suggest that the strategies constitute unnecessary computational effort.

5.5 Evaluation

After the models are trained, they play 20 games against each other to evaluate playing strength. The models also record the number of simulations executed from MCTS during training as well as test-play to enable comparisons between computational effort spent.

6 Results

Of the resources provided by the HPC,

- 1 NVIDIA A100 GPU,
- 32 AMD EPYC CPUs, and
- 150 GB of memory

are requested for each individual training run. For the duration of 24 hours, 28 threads simultaneously play games to collect training data. The fixed number of simulations is 200 either as the hard number for the base version or as reference for the heuristic methods. Informal testing showed that all models showed improvement during training.

	Played games	States	Simulations	Average sim's per state
vanilla_az	72353.0	8057947.0	1611589400.0	200.0
top_k_az	50757.0	5584722.0	1038874885.0	186.02
top_bot_k_az	43913.0	4538558.0	1092512358.0	240.72
rl_az	81288.0	5397453.0	1131621377.0	209.66
rand_az	46501.0	5155559.0	1033063100.0	200.38

Table 1: Results of training

Table 1 summarises the self-play training statistics across the five AlphaZero variants augmented with the heuristics. For the number of played games and states, the highest number is most relevant; in contrast, the number of simulations and the average number of simulations should be small.

The largest amount of games was played by **rl_az** with 81, 288 games. Furthermore, the highest number of states was researched by **vanilla_az** with 8.1 mio. states. The lowest number of simulations was executed by **rand_az** with 1.033 mio. simulations. The lowest number of

average simulations was done by `top_k_az` with 186. Regarding the goal of increasing the number of games as much as possible, `rl_az` seems very successful. However, this makes the relatively smaller number of simulations more surprising.

The ratio of the greatest difference in simulation number is between `vanilla_az` and `rand_az` and is about 56%. The second-highest simulation count is from `rl_az` and is 42.41% lower than `vanilla_az`'s. That base `vanilla_az` has such a large amount of simulations more may indicate that the overhead from every heuristic is larger than expected. This interpretation is also supported by the fact that `rand_az` has approximately the same amount of simulations per state as `vanilla_az` as this indicates that on average the same amount of time was spent on every node's tree search aside from the generation of a random number.

It stands to reason that since `top_bot_k_az` has the highest average number of simulations per move, this causes the search of states and consequently the generation of games to stall. Since `rl_az` has a high number of games but compared to `vanilla_az` a low number of states, it seems apparent that the training games of `rl_az` are shorter than for other versions.

B/W	<code>vanilla_az</code>	<code>top_k_az</code>	<code>top_bot_k_az</code>	<code>rl_az</code>	<code>rand_az</code>
<code>vanilla_az</code>	0	5	3	6	7
<code>top_k_az</code>	3	0	2	7	9
<code>top_bot_k_az</code>	9	6	0	10	9
<code>rl_az</code>	9	5	3	0	2
<code>rand_az</code>	5	2	3	9	0

Table 2: Results of matched playing from the perspective of black.

	<code>vanilla_az</code>	<code>top_k_az</code>	<code>top_bot_k_az</code>	<code>rl_az</code>	<code>rand_az</code>
<code>vanilla_az</code>	0	12	4	7	12
<code>top_k_az</code>	8	0	6	12	17
<code>top_bot_k_az</code>	16	14	0	17	16
<code>rl_az</code>	13	8	3	0	3
<code>rand_az</code>	8	3	4	17	0

Table 3: Results of matched playing from the perspective of white.

Table 2 presents how many games a model in the row playing as black has won against the model in the column playing as white. In contrast, Table 3 presents the total number of wins in

each matchup either as black or white. Values below the diagonal are won by the row-model. Values above the diagonal are won by the row-model.

Model	Wins	Simulations
vanilla_az	35	1062333
top_k_az	43	800564
top_bot_k_az	63	1099080
rl_az	27	672033
rand_az	32	1251804

Table 4: Number of simulations per model over all games.

Table 4 presents the number of matches won over all 80 matches against the other versions. It is evident from these tables that top_bot_k_az is the most successful of models with 63 total wins and 1062333.0 simulations. The least successful version is rl_az with 27 total wins with the fewest number of simulations of 672033.0. Curiously, rand_az has the highest number of simulations and the second-lowest number of wins with 32. Positively, top_k_az has the second-lowest number of simulations executed during the games with 800564.0 but also the second-highest number of wins with 43. Comparing this to vanilla_az and rand_az, which both have fewer games won and more simulations executed, it is quite successful.

7 Discussion

It is not clear how rl_az came to execute 12.35% more games than vanilla_az even though, fewer simulations have been executed and the average amount of simulations is higher. There are many issues that can be raised against this project. First, only one training run for each variation of AlphaZero has been executed, thus reproducibility of these results has not been verified here. Furthermore, training data was sparse because of the short time of training and less hardware than is required for stable training. This also exacerbates the fact that at high levels of play even small mistakes can quickly lead to catastrophic results. As none of these versions of AlphaZero can be expected to have reached high capability, the results may change on reproduction with increased time and hardware limits. Additionally, the number of

simulations that are executed in each board state should be kept track of to accurately judge how the heuristics respond to board positions.

If despite these weaknesses these results are taken at face value, they show little positive results. Not only do the heuristics research far fewer states but most heuristics also show an increased number average simulations per state.

However, The training games and runs shows positive results for `top_k_az`. First, it has lower number of average simulations per state than any other version in the training runs. Second, it has won the second-most games of any version. Next, it has also executed the second-lowest number of simulations over all test games of any version. Further experimentation on this heuristic could include changing the window of consideration and a warm up phase, in which the heuristics is not applied so that the model may learn a baseline understanding of the environment first. Lastly, the uncertainty function may be made more or less aggressive by changing the `sp` parameter.

As `rand_az` is the second-weakest player and `top_k_az` the second-strongest and the direct exchange won by `top_k_az` with a record of 17 : 3, `top_k_az` seems far more capable than `rand_az`. Thus, it can be assumed that the intent does matter in manipulating the number of simulations, as long as the number of simulations to which `top_k_az` is compared is not too high, as the comparison with `top_bot_k_az` shows.

Furthermore, I think that `rl_az` still holds promise. Because of the aforementioned issues of this thesis, an additional head to the network may more noise than can reasonably be offset by the network. Especially in the beginning of training, this may cause additional confusion. However, this may be resolved by again using warm-up phases to relieve noise on gameplay itself in early stages of training and gradually start applying uncertainty to calculating new simulation budgets and reintroducing the training of the uncertainty head into the loss.

8 Conclusion

This thesis has presented the historical and theoretical background of AlphaZero systems. It has then presented methods which were expected to improve such systems' efficiency. This goal has not been reached by many of the heuristics with the exception being `top_k_az`. Next, the results of the training runs and the test games were analysed.

Of all heuristics, `top_bot_k_az` achieved the highest number of wins overall, demonstrating that larger simulation budgets will mostly generate better results. In contrast, `rl_az` proved to be a surprising result as, despite the high number of played games it has a low number of simulations, which indicates a short average game length, it has not learned to learn the game effectively.

Notably, the baseline `vanilla_az` remained competitive throughout, achieving comparable results to more complex variants while being the least expensive to train per unit time. This suggests that the overhead introduced by most heuristics outweighs their benefits, at least within the training duration examined here. The overarching finding of this thesis is therefore that simplicity is competitive with complexity in this setting.

This thesis could not produce conclusive evidence pointing to a positive answer to the hypothesis. Further research and experimentation is needed to answer the following questions: Are these results reproducible? Do heuristics agree on the certainty of a situation? And what effects does changing the window of consideration have on `top_k_az` and `top_bot_k_az`?

Bibliography

- [1] K. G. von Windisch, *Inanimate Reason; or, a Circumstantial Account of That Astonishing Piece of Mechanism, M. de Kempelen's Chess-Player*. S. Bladon, 1784.
- [2] A. M. Turing, “Digital Computers Applied to Games,” in *Faster Than Thought: A Symposium on Digital Computing Machines*, B. V. Bowden, Ed., Pitman, 1953.
- [3] IBM, “Deep Blue.” [Online]. Available: <https://www.ibm.com/history/deep-blue>
- [4] IBM, “Stockfish Blog.” [Online]. Available: <https://stockfishchess.org/blog/2026/stockfish-18/>
- [5] DeepMind, “AlphaGo.” [Online]. Available: <https://deepmind.google/research/alphago/>
- [6] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, Jan. 2016, doi: 10.1038/nature16961.
- [7] AI Index Steering Committee, “Artificial Intelligence Index Report 2025,” Stanford Institute for Human-Centered AI (HAI), Apr. 2025. [Online]. Available: <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- [8] A. E. Elo, *The Rating of Chessplayers, Past & Present*, Reprint. Ishi Press International, 1978. [Online]. Available: <https://archive.org/details/ratingofchesspla0000elo>
- [9] “ManyFaces.” [Online]. Available: <https://www.cgl.ucsf.edu/go/Programs/ManyFaces.html>
- [10] Martin Müller, “Computer Go,” *Artificial Intelligence*, vol. 134, no. 1–2, pp. 145–179, Jan. 2002, doi: 10.1016/S0004-3702(01)00121-7.
- [11] “Monte Carlo Tree Search in Go.” [Online]. Available: <https://webdocs.cs.ualberta.ca/~mmueller/ps/2013/2013-gochapter-preprint.pdf>
- [12] *GnuGo*. [Online]. Available: <https://www.gnu.org/software/gnugo/devel.html>
- [13] Rémi Coulom, “Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search,” May 2006, *Springer, Berlin, Heidelberg*. doi: 10.1007/978-3-540-75538-8_7.
- [14] S. Gelly, L. Kocsis, D. Silver, and C. Szepesvári, “The Grand Challenge of Computer Go: Monte Carlo Tree Search and Extensions,” vol. 55, no. 3, pp. 106–113, Mar. 2012, doi: 10.1145/2093548.2093574.

- [15] D. Silver *et al.*, “Mastering the game of Go without human knowledge,” *Nature*, vol. 550, pp. 354–359, Oct. 2017, doi: 10.1038/nature24270.
- [16] D. Silver *et al.*, “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.” doi: 10.48550/arXiv.1712.01815.
- [17] J. Schrittwieser *et al.*, “Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model.” doi: 10.48550/arXiv.1911.08265.
- [18] L. Kocsis and C. Szepesvári, “Bandit Based Monte-Carlo Planning,” 2006. doi: 10.1007/11871842_29.
- [19] I. Danihelka, A. Guez, J. Schrittwieser, and D. Silver, “Policy Improvement by Planning with Gumbel,” 2022.
- [20] D. J. Wu, “Accelerating Self-Play Learning in Go,” Nov. 10, 2020. doi: 10.48550/arXiv.1902.10565.
- [21] *Leela-zero*.
- [22] L.-C. Lan, M.-Y. Tsai, T.-R. Wu, I.-C. Wu, and C.-J. Hsieh, “Learning to Stop: Dynamic Simulation Monte-Carlo Tree Search.” doi: 10.48550/arXiv.2012.07910.
- [23] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [24] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 5, Nov. 1958, doi: 10.1037/h0042519.
- [25] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.
- [26] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” in *Proceedings of the IEEE*, Nov. 1998, pp. 2278–2324. doi: 10.1109/5.726791.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, Jun. 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

- [28] M. G. Kendall and J. D. Gibbons, “Tied Ranks,” in *Rank Correlation Methods*, 5th ed., 1990, pp. 40–60.
- [29] J. Tromp and G. Farnebäck, “Combinatorics of Go,” 2006, *Springer Berlin, Heidelberg*. doi: 10.1007/978-3-540-75538-8_8.
- [30] Michael Hu, *Alpha Zero: A PyTorch implementation of DeepMind's AlphaZero agent*. [Online]. Available: https://github.com/michaelnny/alpha_zero